Nikhil Savant

312-989-8206 | nsavant033@gmail.com | linkedin.com/in/nikhil-savant | U.S. Citizen

Education

University of Illinois Urbana-Champaign — Urbana-Champaign, IL

Bachelor of Science in Computer Science and Economics — *Expected Graduation: 2028*

Coursework: Database Systems, Algorithms and Models of Computation, Computer Systems, Data Structures

Technical Skills and Clubs

Languages: Java, Python, C++, C, Go, TypeScript, JavaScript, R, HTML/CSS, Swift, SQL
Frameworks: LitJS, Node.js, Spring Boot, NextJS, React, Angular, React Native, LangChain
Developer Tools: Linux, CoreML, AWS, Jupyter, Git, MongoDB, Docker, Figma, Nginx, Kubernetes
Libraries: Pandas, NumPy, OpenCV, Tkinter, SK-Learn, TensorFlow
Clubs: UIUC ACM Math and Algorithms Member, UIUC Engineering Council Activities Coordinator

Work

Cisco, San Jose, CA — May - Aug 2026

Software Engineering Intern

- Built an AI Agent that connects users' natural language questions with results from **9+** live infrastructure tools, securing AI-initiated workflows with an authorization model that enforces caller-scoped delegated identity and least-privilege RBAC at the controller level
- Engineered a repeatable RAG ingestion pipeline, processing **13,000+** documents and **2,300+** text chunks into an embedded vector database to enable fast, semantic retrieval and deterministic source citation
- Created a typed AI-agent harness and evaluation framework across 5 bounded planning stages, hardening the model against prompt brittleness using paraphrase regression suites and fail-closed result verifiers across our infrastructure tools.
- Developed reliability controls and a verifier-gated conversation-state cache, implementing end-to-end trace propagation and automated release quality gates to accelerate incident diagnosis

Algo Analytics — Jun - Oct 2025

Full Stack Engineer Intern

- Developed a TypeScript based backend integration layer to connect the React Native client with financial API's, aggregating real-time stock data and reducing client-side parsing times by **30%**
- Implemented RESTful API controllers to serialize and sanitize high-throughput financial data streams, successfully processing **100,000+** data points daily while achieving a **300ms** API response time
- Developed scalable backend services that utilized JWT authentication and Axios interceptors for secure session management, optimizing JSON payload sizes by **20%** and ensuring zero-downtime token refresh cycles

StellarPay — Jun - Aug 2025

Software Engineering Intern

- Built a GPT-powered financial assistant in TypeScript/Node.js, leveraging a Retrieval-Augmented Generation (RAG) system with engineered prompts to deliver precise, context-aware insights
- Refactored front-end codebase and components improving component reusability and load times leading to a reduction of code duplication by **25%** and a reduction in load times by **20%**
- Enhanced retrieval precision by **40%** using AWS Textract for document ingestion and OpenAI Embeddings with Pinecone for high-performance semantic search
- Implemented end-to-end security protocols across AWS services, including IAM role-based access control, encrypted data storage with AWS KMS (AES-256), and API Gateway authorization layers using JWTs to safeguard sensitive user data

Projects

Custom Git Server

- Created a custom Git server in Go capable of remote cloning, fetching, and pushing via the Smart HTTP and SSH protocols, allowing direct handling of TCP/IP socket connections
- Developed a secure authentication pipeline integrating EdDSA SSH key validation and JWTs, ensuring strict process isolation and role-based access control
- Designed an efficient streaming parser to read raw Git object packs and parse DAGs, capping buffer memory at under **30MB** for large repository transfers

Container Runtime

- Developed a custom Go container runtime using Linux namespaces (PID, NET, MOUNT), capability restrictions, and seccomp filtering to isolate and execute application workloads
- Enforced CPU, memory, and process limits with cgroup v2, validating 64 concurrent workloads across 1,280 successful test requests with **2.06** ms p95 response latency on a 4-vCPU Linux VM
- Implemented shared OverlayFS image layers, achieving **5.16** ms p95 warm startup and reducing allocated layer storage by **98.3%** versus duplicated base images across 64 instances.